Detect and Avoid Problematic Content

Overview

This section focuses on **identifying and preventing**:

- Redundant content
- Infinite crawling loops (spider traps)
- Low-value or noisy data

Goal:
Ensure crawler stores only **useful, unique, and safe content**

1. Redundant Content

Problem

- **~30% of web pages are duplicates**
- Same content may exist at:
    - Different URLs
    - Slight variations

Impact

- Wastes:
    - Storage
    - Bandwidth
    - Processing power
- Slows down crawling efficiency

Solution

- Use **hashing / checksums**

How it works:

1. Generate hash of page content
2. Compare with existing hashes
3. If match found → content already exists → discard

Key Benefit

- Fast duplicate detection
- Reduces unnecessary storage and processing

2. Spider Traps

What is a Spider Trap?

- A web structure that causes crawler to enter **infinite loop**

Example

```
None

www.spidertrapexample.com/foo/bar/foo/bar/foo/bar/…
```

Problem

- Crawler keeps generating:

    - Infinite URLs
- Leads to:
    - Resource exhaustion
    - Wasted crawl cycles

Detection Challenges

- No universal algorithm exists
- Hard to automatically detect all traps

Solutions

1. URL Length Limitation

- Set **maximum URL length**
- Prevents infinitely growing paths

2. Abnormal Pattern Detection

- Identify websites with:
    - Unusually high number of generated pages

3. Manual Intervention

- Identify suspicious domains
- Apply:
    - Blacklisting
    - Custom URL filters

Key Insight

- Combination of:
    - Automated heuristics

    - Manual validation

3. Data Noise

Problem

Some web content has **little or no value**, such as:

- Advertisements
- Spam URLs
- Code snippets
- Irrelevant or junk data

Impact

- Pollutes dataset
- Reduces quality of search/indexing
- Increases storage cost

Solution

- Apply **content filtering mechanisms**

Techniques:

- URL filtering
- Content-based filtering
- Blacklists / heuristics

Goal

- Keep only:
    - Relevant
    - High-quality
    - Meaningful content

Key Takeaway

To build an efficient crawler:

- Remove **duplicate data** → hashing
- Avoid **infinite loops** → spider trap handling
- Filter **low-value content** → noise reduction

Ensures:

- Better performance
- Efficient storage
- High-quality indexing